



## A HEURISTIC ALGORITHM FOR THE DRONE RURAL POSTMAN PROBLEM

AILING XIE<sup>✉1</sup>, KEIGO MIYAGAWA<sup>✉2</sup>, WEI WU<sup>✉2</sup>  
AND MUTSUNORI YAGIURA<sup>✉1</sup>

<sup>1</sup>Nagoya University, Japan

<sup>2</sup>Shizuoka University, Japan

(Communicated by Changjun Yu)

**ABSTRACT.** In this paper, we consider the drone rural postman problem (DRPP). Unlike a vehicle in the traditional rural postman problem, a drone can travel directly between any two points without following the edges of the graph. Thus, a drone can start (or restart) and end (or pause) the service of an edge at any two points of it, and an edge can be serviced separately (e.g., from an end point to the middle point in the first visit and the remaining half in the second visit). This property implies that a lower cost solution may be obtained by using a drone. In DRPP instances, the input data are represented by approximating each edge that have to be traversed by a polygonal chain with a finite number of vertices, allowing the drone to enter and exit each edge at these vertices. For this problem, we propose a heuristic algorithm consisting of two phases. In the first phase, we reduce the size of the original graph by considering the degree of each vertex and the cheapest travel costs between connected components. In the second phase, we solve the rural postman problem on the resulting graph. We show that the proposed two-phase heuristic has a performance guarantee of ratio 2. The experimental results on benchmark instances show that the proposed algorithm outperforms other existing algorithms.

**1. Introduction.** Given a graph and a set of *required edges* to be serviced, the rural postman problem (RPP) aims to find a (closed) route with the minimum cost such that all the required edges are traversed. The RPP is known to be  $\mathcal{NP}$ -hard [5, 6], whereas a well-known special case, the Chinese postman problem (CPP) on undirected graphs and on directed graphs, is polynomially solvable [4, 6]. The classical RPP has broad real-world applications such as street snow cleaning, school bus routing, solid waste collection, and gas pipeline inspection [5].

In this paper, we consider the drone rural postman problem (DRPP), in which a drone is used instead of a vehicle. The use of a drone causes a significant change in the traditional RPP model [1]. Aerial drones are more efficient (from the view point

---

2020 *Mathematics Subject Classification.* Primary: 90C27, 90C10; Secondary: 90B06.

*Key words and phrases.* Combinatorial optimization, drone rural postman problem, heuristic algorithm, rural postman problem, drone routing.

The first author is supported by [Graduate Program for Lifestyle Revolution based on Transdisciplinary Mobility Innovation].

\*Corresponding author: Ailing Xie.

of traveling cost) than overland traffic because there is no need to consider complicated road conditions, and drones can fly fast and directly to achieve a straight-line route between two points. A recent survey [7] shows that there are many different types of drones available. Longer-range drones now can fly over 100 km. In this paper, we assume that a drone is able to traverse and finish the services of all the required edges, that is, the DRPP is uncapacitated.

In the classical RPP, a vehicle is not allowed to travel between two vertices if there is no edge between them. On the other hand, a drone for the DRPP can fly between any two points, which are not necessary to be the end vertices of an edge. This property implies that the required edges need to be defined more precisely according to the shape of the edges on the map, since some of them may be irregular curves. To handle such curves practically, Campbell et al. [1] proposed a model in which each required edge is approximately represented by polygonal chain with a finite number of vertices, allowing the drone to enter and exit each edge only at these vertices. In this paper, in designing our algorithm and conducting computational experiments, we use the same assumption for the input data of the problem. That is, we are given a set of required edges represented by polygonal chains, and the costs between all pairs of vertices on the polygonal chains. The DRPP can be considered as an RPP on a complete graph in which the vertex set consists of all the vertices on polygonal chains.

For example, Figure 1 depicts the required edges that must be serviced. Figure 2 shows the instance after approximating each curve by a polygonal chain. Figure 3 depicts an optimal solution to the corresponding instance of Figure 2, where a dashed red line represents an *unrequired traversal* (deadhead) by the drone.

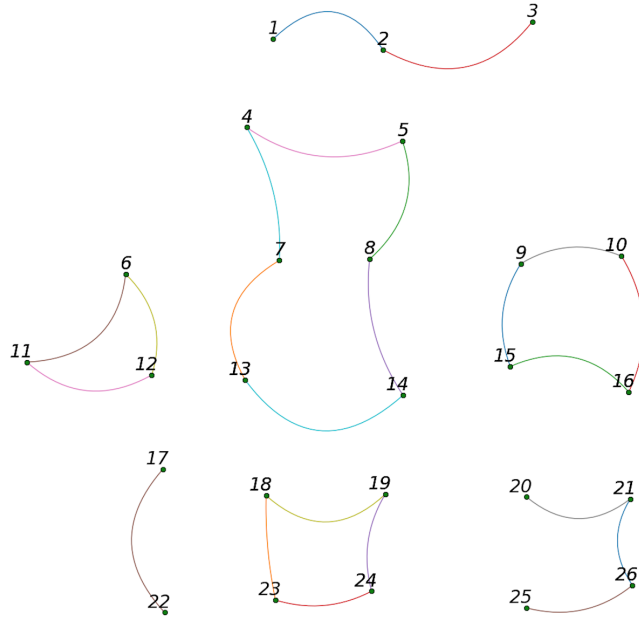


FIGURE 1. Required edges of instance DroneRPP66

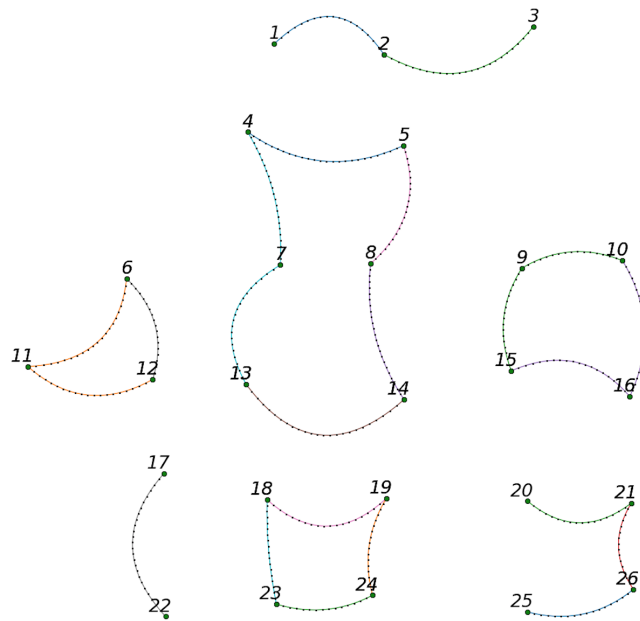


FIGURE 2. Approximation by polygonal chains

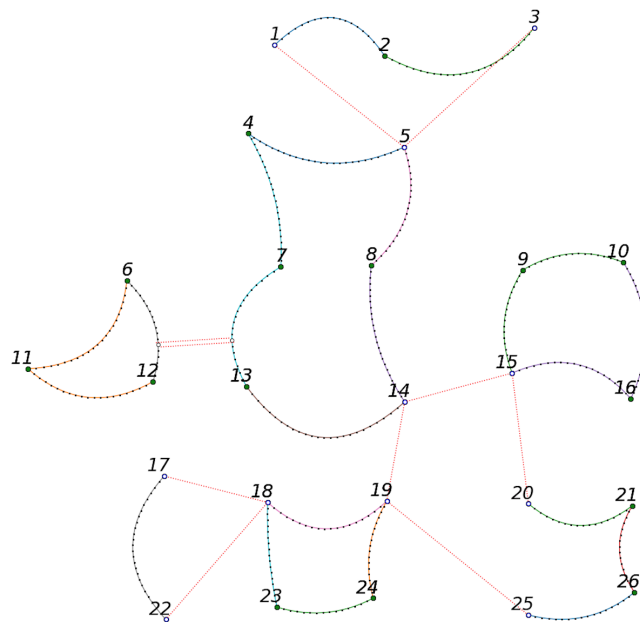


FIGURE 3. An optimal route for instance DroneRPP66

For the DRPP, the cost of a route is equal to the sum of the costs of all the required edges (service cost included) and those of the other edges on the route (deadheads).

In this paper, we propose a heuristic algorithm for the DRPP. It first generates candidate edges for deadheads by considering for the graph induced by the set of required edges, the degree of each vertex and the cheapest travel costs between connected components. It then solves the RPP considering these candidate edges for deadheads instead of the whole complete graph. We first prove that this proposed algorithm is guaranteed to obtain a solution whose objective value is within twice the optimal value. Then, we conduct computational experiments to compare the results of the proposed algorithm with those of existing algorithms in [1] and [3].

## 2. Problem description.

**2.1. Rural postman problem.** Let  $G = (V, E)$  be an undirected graph, where  $V = \{1, 2, \dots, n\}$  is a vertex set and  $E$  is an edge set. We define  $E_R (\subseteq E)$  as the set of edges that must be *serviced* by the vehicle. Edges in  $E_R$  are called *required edges*, and those in  $E \setminus E_R$  are called *unrequired edges*. Each edge  $\{i, j\} \in E$  has a cost  $c_{ij}$  ( $\geq 0$ ) incurred when the vehicle passes through it, and each edge  $\{i, j\} \in E_R$  has an additional service cost  $c'_{ij}$  ( $\geq 0$ ) incurred when the service is provided, where  $c_{ij} = c_{ji}$  and  $c'_{ij} = c'_{ji}$  hold for all  $\{i, j\} \in E$ . Any edge in  $E$  can be *traversed* any number of times. We call the first traversal of a required edge a *required traversal*, and the others (i.e., the second and later traversals of every required edge and all traversals of unrequired edges) *unrequired traversals* or *deadheads*. Then, cost  $c_{ij} + c'_{ij}$  is incurred for the required traversal of each edge  $\{i, j\} \in E_R$ , and cost  $c_{ij}$  is incurred for each unrequired traversal of edge  $\{i, j\} \in E$ . The RPP searches for a closed route with the minimum total cost, such that each required edge is traversed. Note that for any feasible solution, the total cost of the required traversals is a fixed value. Hence, the RPP can be considered as a problem that aims to find a route that minimizes the total cost of deadhead traversals. This problem (on undirected graphs) remains  $\mathcal{NP}$ -hard even if  $c_{ij} = 1$  for all  $\{i, j\} \in E$ , and so does its directed counterpart [6]. Some papers consider the depot at which the route must start and end, but we do not consider it because it can be represented by a required edge with no costs (i.e., if  $i$  is the depot, we can add its copy  $i'$  and set  $c_{ii'} = c'_{ii'} = 0$  and  $c_{i'j} = c_{ij}$  for all  $j$ ). For convenience, we define  $V_R$  as the vertex set consisting of the end vertices of all edges in  $E_R$ . Without loss of generality, we assume that vertex 1 is in  $V_R$ .

The Chinese postman problem on undirected graphs is a well-known special case of the RPP, in which  $E_R = E$ . The CPP on undirected graphs can be transformed to a problem that searches for a minimum-cost edge set  $E' \subseteq E$  such that the graph induced by  $E' \cup E_R$  has an Eulerian circuit, which becomes the minimum cost perfect matching problem and can be solved in polynomial time. A similar argument applies to the directed case. However, the CPP is known to be  $\mathcal{NP}$ -hard on mixed graphs containing both undirected and directed edges [4, 6].

Let  $x_{ij}$  be a binary variable that takes value 1 if the vehicle traverses from  $i$  to  $j$ , and 0 otherwise. We present an integer programming (IP) model for the RPP as follows:

$$\begin{aligned} \text{minimize} \quad & \sum_{\{i,j\} \in E_R} c'_{ij} + \sum_{\{i,j\} \in E} c_{ij}(x_{ij} + x_{ji}) \end{aligned} \tag{1}$$

$$\begin{aligned} \text{subject to} \quad & \sum_{\substack{j \in V: \\ \{i,j\} \in E}} (x_{ij} - x_{ji}) = 0, \end{aligned} \quad \forall i \in V \tag{2}$$

$$\sum_{\substack{i \in S, j \in V \setminus S: \\ \{i, j\} \in E}} x_{ij} \geq 1, \quad \forall S \subset V, 1 \in S, V_R \not\subset S \quad (3)$$

$$x_{ij} + x_{ji} \geq 1, \quad \forall \{i, j\} \in E_R \quad (4)$$

$$x_{ij}, x_{ji} \in \{0, 1\}, \quad \forall \{i, j\} \in E. \quad (5)$$

The objective function (1) minimizes the total cost. Constraints (2) are flow conservation constraints. Constraints (3) are the subroute elimination constraints. Constraints (4) ensure that all the required edges are traversed at least once. Constraints (5) are the binary constraints for all the variables.

Note that even though an edge  $\{i, j\}$  can be traversed any number of times, variables  $x_{ij}$  and  $x_{ji}$  are bounded from above by 1 in (5), instead of allowing them to take larger values.

**Lemma 1.** *There is an optimal RPP solution in which every edge  $\{i, j\} \in E$  is traversed at most once in each direction.*

*Proof.* Suppose that a route has an edge  $\{i, j\}$  traversed twice (or more) in the direction from  $i$  to  $j$ . Then the route can be represented as  $i \rightarrow j \rightarrow \vec{P}_1 \rightarrow i \rightarrow j \rightarrow \vec{P}_2 \rightarrow i$  with  $\vec{P}_1$  and  $\vec{P}_2$  the sequences of edges immediately after the first and second traversal of  $\{i, j\}$ , respectively, with edges arranged in the order of the route. If  $\{i, j\}$  is in  $E \setminus E_R$  or it is traversed three times or more in the route (including the traversals in the opposite direction), then we can rearrange the route to another feasible route  $i \rightarrow \overleftarrow{P}_1 \rightarrow j \rightarrow \vec{P}_2 \rightarrow i$  without increasing the total cost, removing the two traversals of  $\{i, j\}$ , where  $\overleftarrow{P}_1$  is the sequence of edges in  $\vec{P}_1$  traversed in the opposite direction. Otherwise (i.e.,  $\{i, j\}$  is in  $E_R$ , it is traversed exactly twice in the same direction, and it is not traversed in the opposite direction), we can rearrange the route to another feasible route  $i \rightarrow \overleftarrow{P}_1 \rightarrow j \rightarrow i \rightarrow j \rightarrow \vec{P}_2 \rightarrow i$  without changing the total cost, inverting the direction of one of the two traversals of  $\{i, j\}$ . By repeatedly applying these operations, we will reach a desired route for the following reasons. The number of times the former operation can be applied is at most a half of the number of edge traversals in the initial route because two edge traversals are removed whenever it is applied. The latter operation may create a new pair of traversals of an edge in the same direction; however, this will not happen to those edges to which the latter operation has already been applied because after such an operation to edge  $\{i, j\}$ , traversal  $i \rightarrow j$  comes immediately after  $j \rightarrow i$ , and hence both of them will be included in exactly one of  $\vec{P}_1$  or  $\vec{P}_2$  in later operations. Hence the latter operation can be applied at most  $|E_R|$  times. These indicate that after such a number of repetitions, we will reach a route to which we cannot apply these two operations, implying that no edge is traversed twice or more in the same direction.  $\square$

Lemma 1 implies that  $x_{ij}$  and  $x_{ji}$  can be bounded from above by 1 without losing the optimality.

Due to the exponential number of Constraints (3), a constraint generation framework is known to improve overall performance in solving large-scale instances rather than considering all of them in advance [2].

**2.2. Drone rural postman problem.** In the classical RPP problem, a vehicle is not allowed to travel between two vertices if there is no edge between them. However, if there is an edge between two vertices, the road shape on the map

corresponding to the edge does not affect the representation of the problem. On the contrary, in the drone RPP (DRPP), a drone can directly fly between any two points with a small travel cost. It indicates that a more efficient route may exist by taking advantage of the curved shape of some edges. Campbell et al. introduced the following assumptions and they showed that there are cases in which more efficient routes exist if these assumptions are imposed compared to the case they are not imposed [1].

**Assumption 1.** *After finishing the service of a required edge at one of its vertices, the drone can travel directly to any vertex without following the edges of the given graph  $G$ .*

**Assumption 2.** *The drone may start and end the service of an edge at any two points of it, and each edge can be serviced separately (e.g., from an end point of the edge to the middle point in the first visit, and the remaining half in the second visit).*

To define the problem based on these assumptions, the travel cost must be defined between any two points on the curves of required edges (nb. for every pair of points from a curve and every pair of points from two curves (one from each)). It is also natural to assume that when the required traversal of a required edge  $\{i, j\} \in E_R$  is divided into two or more traversals, the total cost (the sum of travel and service costs) of those traversals is equal to the cost  $c_{ij} + c'_{ij}$  of a required traversal from  $i$  to  $j$  on that edge (with  $c_{ij}$  the travel cost and  $c'_{ij} (\geq 0)$  the service cost of  $\{i, j\}$ ), regardless of how the traversal is divided. To make these points clear, we assume the following Assumptions 3 and 4, which are imposed throughout the remainder of the paper.

**Assumption 3.** *For any pair of points  $u$  and  $v$  on the curves of required edges (they can be on one curve or can be on two distinct curves), the travel cost  $c_{uv}$  is defined and is symmetric (i.e.,  $c_{uv} = c_{vu}$ ).*

**Assumption 4.** *Let  $l (\geq 3)$  be an integer, and  $v_1, v_2, \dots, v_l$  be an arbitrary sequence of points on the curve of a required edge  $\{i, j\} \in E_R$ , with  $v_1 = i$  and  $v_l = j$ , and  $v_2, v_3, \dots, v_{l-1}$  appear in this order along the curve of  $\{i, j\}$  from  $i$  to  $j$ . Then, for all  $h = 1, 2, \dots, l-1$ , the service costs  $c'_{v_h, v_{h+1}}$  are defined, nonnegative (i.e.,  $c'_{v_h, v_{h+1}} \geq 0$ ), and symmetric (i.e.,  $c'_{v_h, v_{h+1}} = c'_{v_{h+1}, v_h}$ ). Moreover, the sum of travel and service costs is equal to the original counterpart (i.e.,  $\sum_{h=1}^{l-1} (c_{v_h, v_{h+1}} + c'_{v_h, v_{h+1}}) = c_{ij} + c'_{ij}$ ).*

In proving some theorems below, we also assume that the travel costs satisfy the triangle inequality.

**Assumption 5.** *For any points  $u, v$  and  $w$  on the curves of required edges (they can be on one curve, or on two or three distinct curves), the travel costs satisfy the triangle inequality  $c_{uv} + c_{vw} \geq c_{uw}$ .*

Assumptions 3–5 are natural and there would be many cost definitions that satisfy them. A typical example in defining travel costs is the Euclidean distance (e.g., to describe the cost of an unrequired traversal with a straight-line flight by a drone). It is also natural to define the cost of a required traversal between two points on the curve of a required edge in such a way that it is proportional to the travel distance between the points along the curve. The service cost between two points

is then defined to be the difference between the costs of required and unrequired traversals between them. Let us consider a simple case in which the costs of required and unrequired traversals, respectively, are equal to such distances. Then for any pair of two points  $u$  and  $v$  on the curve of a required edge,  $c_{uv}$  is the straight-line distance between  $u$  and  $v$ , and  $c_{uv} + c'_{uv}$  is the travel distance between  $u$  and  $v$  along the curve. Because the latter is not shorter than the former, we have  $c'_{uv} \geq 0$ , and it is not difficult to see that such costs satisfy Assumptions 3–5.

Concerning the cost savings from the use of drones, Campbell et al. [1] stated that the following theorem holds.

**Theorem 1** (Campbell et al. [1]). *The ratio between the cost of an optimal postman route (an optimal solution to the problem with neither Assumptions 1 nor Assumption 2) and the cost of an optimal drone route to the problem with only Assumption 1 can be as large as desired.*

For the DRPP, we show an upper bound on the largest possible improvement ratio of introducing Assumption 2. Before showing the details, we first introduce solution representations of the DRPP. A solution (route)  $\sigma$  to the DRPP can be naturally represented as a sequence of edges traversed by the drone, arranged in the order of the traversals. If we consider the multigraph induced by all the edges in  $\sigma$ , the multigraph must be connected, and each vertex has an even degree. On the other hand, a multiset of edges covering all the edges in  $E_R$  can also be regarded as a solution representation if its edge-induced multigraph is connected and the degree of each vertex is even, because such a multigraph always has an Eulerian circuit (drone route), and the corresponding route cost depends only on the multiset of edges. For convenience, for a route  $\sigma$  (a sequence of edges traversed by the drone), we also use the same notation  $\sigma$  to denote the multiset of edges in  $\sigma$ . Based on these observations, we propose the following theorem.

**Theorem 2.** *Suppose that Assumptions 3–5 are satisfied. The cost of an optimal drone route  $\sigma_{A1}^*$  to the problem with only Assumption 1 is at most twice the cost of an optimal drone route  $\sigma_{A1\&A2}^*$  to the problem with both Assumptions 1 and 2.*

*Proof.* For the graph  $G_R = (V_R, E_R)$ , let  $V_{\text{odd}} (\subseteq V_R)$  denote the set consisting of all odd-degree vertices. Let  $\mathcal{C} = \{C_1, C_2, \dots, C_r\}$  be the set consisting of all connected components of  $G_R$ , and  $\mathcal{C}_{\text{even}} (\subseteq \mathcal{C})$  be the subset consisting of connected components with only even-degree vertices.

For the problem with only Assumption 1, consider a feasible route  $\sigma''_{A1}$  obtained by Algorithm 1. In this algorithm, the traveling salesman problem (TSP) in Step 2 is a well-known problem that aims to find a shortest (with respect to the travel cost) possible route that visits every vertex exactly once. The disjoint union (also known as discriminated union) in Step 5 is defined as follows. Let  $E_1$  and  $E_2$  be two multisets of edges. Then their disjoint union  $E_1 \uplus E_2$  is defined to be the multiset obtained by keeping between every pair of vertices, all copies of parallel edges from both multisets (e.g., by renaming edge labels for distinguishing the origin), thus consisting of  $|E_1| + |E_2|$  edges. For example, when  $E_1 = \{e_a, e_b, e_c, e_d\}$  and  $E_2 = \{e_a, e_c\}$ , then  $E_1 \uplus E_2 = \{e_a^{(1)}, e_a^{(2)}, e_b^{(1)}, e_c^{(1)}, e_c^{(2)}, e_d^{(1)}\}$ , as illustrated in Figure 4, in which the superscripts “(1)” and “(2)” are added to the edge labels just to discriminate parallel edges.

The existence of an Eulerian circuit in Step 6 is guaranteed by the following two facts:

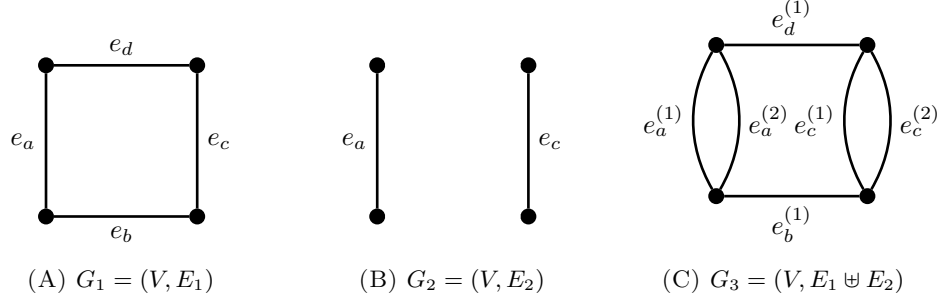


FIGURE 4. Two graphs  $G_1$  and  $G_2$  with edge sets  $E_1$  and  $E_2$ , respectively, and the graph  $G_3$  having edges  $E_1 \uplus E_2$

---

**Algorithm 1** An algorithm for DRPP with Assumption 1

---

**Input:** Graph  $G_R = (V_R, E_R)$ .

**Output:** A feasible route  $\sigma''_{A1}$ .

- 1: Construct a complete graph  $G''$  with vertex set  $V'' = V_{\text{odd}} \cup V_{\text{c-even}}$ , where  $V_{\text{c-even}}$  is the set of  $|\mathcal{C}_{\text{even}}|$  vertices constructed by arbitrarily choosing from every  $C$  in  $\mathcal{C}_{\text{even}}$ , a vertex to be included in  $V_{\text{c-even}}$ .
  - 2: Solve the TSP on graph  $G''$ , and obtain a tour  $\sigma_{\text{tsp}}^*$  that minimizes the total travel cost.
  - 3: Construct a complete graph  $G_{\text{odd}}$  with vertex set  $V_{\text{odd}}$ .
  - 4: Obtain a perfect matching  $M_{\text{odd}}$  on  $G_{\text{odd}}$  that minimizes the total travel cost.
  - 5: Let  $H_{A1}$  be the multigraph induced by the disjoint union of all edges of  $\sigma_{\text{tsp}}^*$ ,  $M_{\text{odd}}$ , and  $E_R$ .
  - 6: Form an Eulerian circuit  $\sigma''_{A1}$  in  $H_{A1}$ .
- 

- All the vertices in  $H_{A1}$  are of even degree, because all the end vertices of the edges involved in  $\sigma_{\text{tsp}}^*$  (resp.,  $M_{\text{odd}} \cup E_R$ ) are of even degree in the graph induced by the set of all edges in  $\sigma_{\text{tsp}}^*$  (resp., the multigraph induced by the disjoint union  $M_{\text{odd}} \uplus E_R$ ).
- The connectivity of  $H_{A1}$  is guaranteed by  $\sigma_{\text{tsp}}^*$ .

Because  $\sigma''_{A1}$  contains all the required edges in  $E_R$  and it does not involve any intermediate points (considered in Assumption 2),  $\sigma''_{A1}$  is a feasible solution to the problem with only Assumption 1.

Next, we show that the ratio between the cost of  $\sigma''_{A1}$  and that of  $\sigma_{A1 \& A2}^*$  is at most 2. Given a route  $\sigma$ , let  $f(\sigma)$  denote the total cost of  $\sigma$  consisting of the total travel cost  $f_{\text{trav}}(\sigma)$  and the total service cost  $f_{\text{serv}}(\sigma)$ , that is,

$$f(\sigma) = f_{\text{trav}}(\sigma) + f_{\text{serv}}(\sigma).$$

Note that by Assumption 4, the total service cost  $f_{\text{serv}}$  is nonnegative for any  $\sigma$ . From the fact that all vertices in  $V''$  are also visited by route  $\sigma_{A1 \& A2}^*$ , we can form a tour  $\sigma'_{\text{tsp}}$  on  $V''$  by visiting them according to the order of  $\sigma_{A1 \& A2}^*$ , skipping those vertices visited more than once. Then, from Assumption 5 (triangle inequality), it holds that

$$f_{\text{trav}}(\sigma'_{\text{tsp}}) \leq f_{\text{trav}}(\sigma_{A1 \& A2}^*). \quad (6)$$



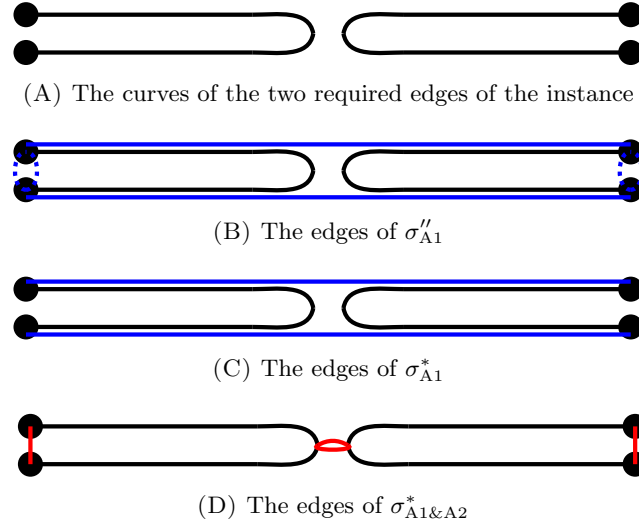


FIGURE 5. A tight example

Because  $\sigma_{\text{tsp}}^*$  is a solution to the TSP on  $V''$  that minimizes the total travel cost  $f_{\text{trav}}$ , we have

$$f_{\text{trav}}(\sigma_{\text{tsp}}^*) \leq f_{\text{trav}}(\sigma'_{\text{tsp}}). \quad (7)$$

From (6)–(7) and  $f_{\text{serv}}(\sigma_{A1 \& A2}^*) \geq 0$ , it holds that

$$f_{\text{trav}}(\sigma_{\text{tsp}}^*) \leq f_{\text{trav}}(\sigma_{A1 \& A2}^*) \leq f(\sigma_{A1 \& A2}^*). \quad (8)$$

Because the disjoint union of all the edges in  $M_{\text{odd}}$  and  $E_R$  is an optimal solution to the relaxed DRPP with both Assumptions 1 and 2 in which the route is not required to be connected, it also holds that

$$f(E_R \uplus M_{\text{odd}}) = \sum_{\{i,j\} \in E_R} (c_{ij} + c'_{ij}) + \sum_{\{i,j\} \in M_{\text{odd}}} c_{ij} \leq f(\sigma_{A1 \& A2}^*). \quad (9)$$

From (8), (9) and the feasibility of  $\sigma''_{A1}$ , route  $\sigma_{A1}^*$  has a cost

$$f(\sigma_{A1}^*) \leq f(\sigma''_{A1}) = f_{\text{trav}}(\sigma_{\text{tsp}}^*) + f(E_R \uplus M_{\text{odd}}) \leq 2f(\sigma_{A1 \& A2}^*). \quad (10)$$

□

**Lemma 2.** *The upper bound of 2 on the ratio in Theorem 2 is tight.*

*Proof.* In order to see that the above ratio of 2 is tight, consider an instance in Figure 5 (A), in which the graph  $G_R = (V_R, E_R)$ , colored in black, has four vertices and two curved edges. The cost of an unrequired traversal between any pair of points is the Euclidean distance, and the cost of a required traversal of each edge  $\{i, j\} \in E_R$  is the Euclidean distance along the curve. For this instance, we observe that

- the route  $\sigma''_{A1}$  obtained by Algorithm 1 is two black curves with six blue edges (two solid lines and four dotted lines in Figure 5 (B)),
- the optimal route  $\sigma_{A1}^*$  under only Assumption 1 is two black curves with two solid blue lines (Figure 5 (C)), and

- the optimal route  $\sigma_{A1\&A2}^*$  under both Assumptions 1 and 2 is two black curves with four red edges (Figure 5 (D)).

It is easy to see that the ratio of the costs of  $\sigma_{A1}^*$  and  $\sigma_{A1\&A2}^*$  can be as close as possible to 2 by setting the nearest points of the two curves extremely close to each other and the two endpoints of each curve extremely close in the vertical direction.  $\square$

It is often not easy to describe the real shape of a continuous curve (e.g., of a curvy road, stream, etc.) and to define the travel cost along it. To practically realize the properties in Assumption 2, the curve of each required edge of the DRPP is approximated and represented by a polygonal chain, allowing the drone to enter and leave the required edge at the vertices of the polygonal chain, as in Campbell et al. [1]. In this representation, a segment of a polygonal chain refers to a required edge, and  $V_R$  and  $E_R$  are composed of all the vertices and segments of polygonal chains, respectively. Hence, the DRPP after approximation can be considered as an RPP in which graph  $G$  is the complete graph on  $V_R$ . For convenience of description, we refer to the problem with curves replaced by polygonal chains as the DRPP and the problem with curves as the *original* DRPP.

Because  $G$  is a complete graph and  $1 \in V_R = V$  holds, the IP model (1)–(5) for the case of the DRPP can be simplified as follows:

$$\text{minimize} \quad \sum_{\{i,j\} \in E_R} c'_{ij} + \sum_{\{i,j\} \in E} c_{ij}(x_{ij} + x_{ji}) \quad (11)$$

$$\text{subject to} \quad \sum_{j \in V \setminus \{i\}} (x_{ij} - x_{ji}) = 0, \quad \forall i \in V \quad (12)$$

$$\sum_{i \in S, j \in V \setminus S} x_{ij} \geq 1, \quad \forall S \subsetneq V, 1 \in S \quad (13)$$

$$x_{ij} + x_{ji} \geq 1, \quad \forall \{i, j\} \in E_R \quad (14)$$

$$x_{ij}, x_{ji} \in \{0, 1\}, \quad \forall \{i, j\} \in E. \quad (15)$$

**3. Heuristic algorithm.** The difficulty in solving the DRPP is that the size of the graph can become very large after approximating curves by polygonal chains. For example, the instance in Figure 1 has 23 curves, but the resulting graph has 463 vertices and 106,953 edges. For the DRPP, we propose a heuristic algorithm consisting of two phases. The first phase reduces the graph and the second phase solves the RPP on the reduced graph.

In the first phase, we generate a graph  $G'$  that is much smaller in size compared to graph  $G$ . Graph  $G'$  is initialized as a graph that contains all the segments in each polygonal chain, that is,  $G' \leftarrow (V_R, E_R)$ . We then consider adding (unrequired) edges to  $G'$  with the following steps:

- I. For every pair of odd-degree vertex  $i$  and connected component  $C$  that does not contain vertex  $i$ , we mark the end vertices of the edge connecting  $i$  and  $C$  with the shortest distance.
- II. For every pair of connected components  $C$  and  $C'$ , we mark the end vertices of the edge connecting  $C$  and  $C'$  with the shortest distance.
- III. For every marked vertex  $i$ , if the degree of  $i$  is odd, we add edges from  $i$  to all the other marked vertices; otherwise, we add edges from  $i$  to all marked vertices in every connected component not containing  $i$ .

In the second phase, we solve the problem defined on the resulting graph  $G'$  using the IP model (1)–(5) presented in Section 2.1.

The pseudocode of the proposed algorithm is shown in Algorithm 2.

---

**Algorithm 2** A heuristic algorithm for DRPP

---

**Input:** Graph  $G = (V_R, E_R)$ .

**Output:** A feasible route  $\sigma_{\text{alg2}}$ .

- 1: Initialize a graph  $G'$  as  $G' \leftarrow G$ .
  - 2: Add unrequired edges to  $G'$  following Steps I–III.
  - 3: Exactly solve the IP model (1)–(5) defined on  $G'$  by an IP solver to obtain a feasible route  $\sigma_{\text{alg2}}$ .
- 

**4. Algorithm analysis.** We demonstrate the performance guarantee of Algorithm 2 by using Theorem 2. From the rules in Steps I to III, it is easy to see that after Step 2 of Algorithm 2, every odd-degree vertex is marked and there is a marked vertex in every connected component. Hence, in Step 2 of Algorithm 1, the set  $V'' = V_{\text{odd}} \cup V_{\text{c-even}}$  can be chosen in such a way that all members are those that will be marked in Step 2 of Algorithm 2 (to be more precise, all vertices in  $V_{\text{odd}}$  will be marked, and  $V_{\text{c-even}}$  can be chosen in that way); even with this choice, the proof of Theorem 2 applies. Suppose that Algorithm 1 chooses such a  $V_{\text{c-even}}$  in Step 1. Then, both  $G''$  and  $G_{\text{odd}}$  constructed in Algorithm 1 are subgraphs of  $G'$  constructed in Algorithm 2. Therefore, the route  $\sigma_{\text{alg2}}$  obtained by Algorithm 2 is at least as good as the route  $\sigma''_{A1}$  obtained by Algorithm 1, and the following theorem holds.

**Theorem 3.** *Suppose that Assumptions 3–5 are satisfied. Then the optimal value to the RPP on the graph  $G'$  constructed in Algorithm 2 is within twice the optimal value to the original DRPP.*

From this theorem, the following corollary is immediate.

**Corollary 4.** *Algorithm 2 has a performance guarantee of ratio 2 when Assumptions 3–5 are satisfied.*

**5. Computational experiments.** All experiments were carried out on a PC with a Xeon E-2286G CPU running at 4.00 GHz, with 64 GB memory, with computations always conducted on a single thread. All algorithms were coded in Python (version 3.8.10). We used Gurobi Optimizer (version 9.5) as an IP solver to solve the model (1)–(5) and (11)–(15) in Section 2. The time limit for each run was set to 3600 seconds.

Two types of benchmark instances, **random** and **even** [1], are used in the experiments. The **even** instances have few odd-degree vertices in graph  $(V_R, E_R)$  and are known to be more difficult to solve than **random** instances. The travel cost  $c_{ij}$  is given by the Euclidean distance according to the coordinates of  $i$  and  $j$ .

Table 1 and 2 show the results obtained by a heuristic algorithm proposed in [1] (“existing heuristic”), by an exact branch-and-bound algorithm proposed in [3] (“existing exact”), by our heuristic algorithm (“proposed”), and by applying Gurobi to the RPP model defined on the complete graph (“Gurobi”). The results of “existing heuristic” and “existing exact” are taken from columns labeled “RPP(3)” and “Drone RPP” of Tables 4 and 5 in [1] (to be more precise, the values in our

tables are calculated from the values in columns “Imp.(%)” and “Opt.” of these tables in [1]). Values in “%gap” are calculated by  $100(UB - LB)/LB$ , where  $UB$  is the objective function value obtained by the corresponding algorithm, and  $LB$  is the optimal value, or a better value between the lower bound provided in [1] and the lower bound obtained by applying Gurobi to the RPP model defined on the complete graph until memory-over occurs (with no time limit). The best gaps are marked in bold. Values in “time” show the CPU times in seconds required to obtain the best solutions, where the values in columns “existing heuristic” and “existing exact” are those on a PC with an Intel Core i7 running at 3.4 GHz with 32 GB memory, and these algorithms were run with a time limit of 3600 seconds. The computation time in column “proposed” contains the time to generate the graph  $G'$  explained in Section 3, which does not exceed 0.5 seconds for every instance and is usually small compared to the time to solve the IP model defined on  $G'$ . Note that “existing heuristic” uses the exact method in [3], applying it to an instance with heuristically reduced candidates of unrequired edges. The notation “t.l.” means that the algorithms failed to exactly solve the instance (in the case of “existing heuristic,” the exact algorithm invoked inside the heuristic method failed to exactly solve the reduced instance) within the time limit, and  $UB$  values are not available in [1] (for such instances, only  $LB$  values are provided in [1]).

For **random** instances, the proposed heuristic algorithm obtained optimal solutions for all the 29 instances whose optimal solutions were known. On the other hand, the existing heuristic algorithm failed to obtain optimal solutions for three instances. For instance DroneRPP8101, for which both existing exact algorithm and Gurobi failed to solve to optimality, both the existing and the proposed heuristic algorithms obtained a solution with the same objective function value that is better than the solution value obtained by Gurobi.

For **even** instances, we also confirmed that the proposed heuristic algorithm obtained optimal solutions for all the instances (8 out of 15) whose optimal solutions were known. Furthermore, the proposed heuristic algorithm reported best gaps for all the other instances. Compared to the existing heuristic algorithm, the proposed algorithm obtained a solution with a better objective function value for all the even instances.

To better compare the existing heuristic algorithm with the proposed algorithm, we implemented the existing heuristic algorithm using the same programming language (Python) and the same RPP solver (Gurobi) as the proposed algorithm. As mentioned above, the existing heuristic is also a two-phase method that generates in the first phase, a graph by heuristically reducing unrequired edges and then applies in the second phase, an exact method to the RPP on the graph generated in the first phase. The time spent for the first phase is usually very small, and hence implementation skills will have a small influence on the computational results. Tables 3 and 4 show the results obtained by the proposed algorithm and the existing heuristic algorithm implemented by us, with the same PC and the same time limit of 3600 seconds. All notations have the same meaning as in Tables 1 and 2, and values in column “#vertices” show the number of vertices in the graph generated in the first phase. The computation time in column “time” contains the time to generate the graph. For the existing heuristic, the time to generate the graph in the first phase does not exceed 0.1 seconds for every instance. Values in bold indicate that the corresponding algorithm obtained a better solution within the time limit, or obtained a solution with the same cost but in less time.

TABLE 1. Results for **random** instances

| instance      | LB       | existing heuristic |                   | existing exact |                   | proposed    |                   | Gurobi      |                   |
|---------------|----------|--------------------|-------------------|----------------|-------------------|-------------|-------------------|-------------|-------------------|
|               |          | %gap               | time <sup>†</sup> | %gap           | time <sup>†</sup> | %gap        | time <sup>‡</sup> | %gap        | time <sup>‡</sup> |
| DroneRPP561   | 4347.09  | <b>0.00</b>        | 0.30              | <b>0.00</b>    | 8.17              | <b>0.00</b> | 0.29              | <b>0.00</b> | 18.53             |
| DroneRPP562   | 5123.62  | <b>0.00</b>        | 0.22              | <b>0.00</b>    | 3.26              | <b>0.00</b> | 0.27              | <b>0.00</b> | 51.01             |
| DroneRPP581   | 5929.64  | <b>0.00</b>        | 0.37              | <b>0.00</b>    | 9.62              | <b>0.00</b> | 0.65              | <b>0.00</b> | 45.26             |
| DroneRPP582   | 7274.21  | <b>0.00</b>        | 0.94              | <b>0.00</b>    | 14.75             | <b>0.00</b> | 0.56              | <b>0.00</b> | 57.76             |
| DroneRPP5101  | 6592.95  | 0.02               | 0.35              | <b>0.00</b>    | 36.53             | <b>0.00</b> | 1.07              | <b>0.00</b> | 72.45             |
| DroneRPP5102  | 10259.65 | <b>0.00</b>        | 0.83              | <b>0.00</b>    | 19.98             | <b>0.00</b> | 1.11              | <b>0.00</b> | 367.57            |
| DroneRPP661   | 4684.62  | <b>0.00</b>        | 0.34              | <b>0.00</b>    | 4.06              | <b>0.00</b> | 0.34              | <b>0.00</b> | 16.02             |
| DroneRPP662   | 5298.72  | <b>0.00</b>        | 0.30              | <b>0.00</b>    | 13.44             | <b>0.00</b> | 0.41              | <b>0.00</b> | 15.80             |
| DroneRPP681   | 6391.92  | <b>0.00</b>        | 1.00              | <b>0.00</b>    | 117.08            | <b>0.00</b> | 0.76              | <b>0.00</b> | 97.65             |
| DroneRPP682   | 8797.11  | <b>0.00</b>        | 0.81              | <b>0.00</b>    | 20.50             | <b>0.00</b> | 0.91              | <b>0.00</b> | 245.50            |
| DroneRPP6101  | 9258.50  | 0.01               | 0.53              | <b>0.00</b>    | 23.68             | <b>0.00</b> | 9.63              | <b>0.00</b> | 320.32            |
| DroneRPP6102  | 12271.33 | <b>0.00</b>        | 1.81              | <b>0.00</b>    | 163.56            | <b>0.00</b> | 3.00              | <b>0.00</b> | 2830.16           |
| DroneRPP771   | 6654.00  | <b>0.00</b>        | 1.25              | <b>0.00</b>    | 744.85            | <b>0.00</b> | 5.07              | <b>0.00</b> | 450.74            |
| DroneRPP772   | 10441.46 | <b>0.00</b>        | 0.94              | <b>0.00</b>    | 5.75              | <b>0.00</b> | 0.88              | <b>0.00</b> | 185.02            |
| DroneRPP791   | 10039.18 | <b>0.00</b>        | 1.78              | t.l.           | t.l.              | <b>0.00</b> | 8.35              | <b>0.00</b> | 1461.35           |
| DroneRPP792   | 11946.27 | 0.01               | 0.97              | <b>0.00</b>    | 55.77             | <b>0.00</b> | 3.13              | <b>0.00</b> | 888.95            |
| DroneRPP7101  | 11019.16 | 0.01               | 1.78              | <b>0.00</b>    | 177.04            | <b>0.00</b> | 4.95              | 0.04        | 567.25            |
| DroneRPP7102  | 13425.04 | <b>0.00</b>        | 0.72              | <b>0.00</b>    | 27.05             | <b>0.00</b> | 1.85              | <b>0.00</b> | 480.76            |
| DroneRPP881   | 10263.85 | <b>0.00</b>        | 0.88              | <b>0.00</b>    | 41.93             | <b>0.00</b> | 3.88              | <b>0.00</b> | 631.35            |
| DroneRPP882   | 12538.78 | <b>0.00</b>        | 11.30             | <b>0.00</b>    | 217.18            | <b>0.00</b> | 2.61              | <b>0.00</b> | 744.61            |
| DroneRPP891   | 10911.38 | 0.01               | 2.41              | <b>0.00</b>    | 3266.83           | <b>0.00</b> | 6.20              | <b>0.00</b> | 1590.72           |
| DroneRPP892   | 12519.73 | <b>0.00</b>        | 0.53              | <b>0.00</b>    | 28.43             | <b>0.00</b> | 2.28              | <b>0.00</b> | 534.00            |
| DroneRPP8101  | 12414.48 | <b>0.66</b>        | 5.36              | t.l.           | t.l.              | <b>0.66</b> | 33.68             | 0.91        | 2925.44           |
| DroneRPP8102  | 16020.01 | <b>0.00</b>        | 1.95              | <b>0.00</b>    | 49.41             | <b>0.00</b> | 3.27              | 0.01        | 2233.96           |
| DroneRPP991   | 12118.63 | <b>0.00</b>        | 0.96              | <b>0.00</b>    | 528.07            | <b>0.00</b> | 6.05              | <b>0.00</b> | 1977.60           |
| DroneRPP992   | 16365.53 | <b>0.00</b>        | 2.66              | <b>0.00</b>    | 159.89            | <b>0.00</b> | 3.81              | 0.07        | 2977.23           |
| DroneRPP9101  | 13282.73 | <b>0.00</b>        | 3.53              | <b>0.00</b>    | 46.13             | <b>0.00</b> | 6.04              | <b>0.00</b> | 1204.49           |
| DroneRPP9102  | 16636.71 | 0.01               | 4.20              | <b>0.00</b>    | 830.98            | <b>0.00</b> | 16.59             | 0.58        | 1545.82           |
| DroneRPP10101 | 14997.03 | <b>0.00</b>        | 1.28              | <b>0.00</b>    | 129.41            | <b>0.00</b> | 6.18              | 0.64        | 1493.77           |
| DroneRPP10102 | 17532.11 | 0.01               | 13.13             | <b>0.00</b>    | 1284.55           | <b>0.00</b> | 5.34              | 0.28        | 2926.56           |

<sup>†</sup>The CPU time on a PC with an Intel Core i7 running at 3.4 GHz with 32 GB memory.

<sup>‡</sup>The CPU time on a PC with Xeon E-2286G CPU running at 4.00 GHz, with 64 GB memory, with computations always conducted on a single thread.

From Tables 3 and 4, we observe that the proposed algorithm generated smaller graphs in the first phase for all the tested instances. For **random** instances, the proposed algorithm obtained solutions with lower costs for 7 out of 30 instances. For the remaining 23 instances, two algorithms obtained solutions with the same costs, but the proposed algorithm required less CPU time for 16 out of 23 instances. For **even** instances, the proposed algorithm obtained better solutions for all the 15 instances. The results in Tables 3 and 4 indicate that the graph reduction in the proposed method is very effective, obtaining solutions with equal or lower costs for all the tested instances, even though the numbers of vertices in the RPP are smaller.

**6. Conclusion.** We considered the drone rural postman problem (DRPP), which can be modeled as a classical rural postman problem (RPP). However, due to approximation accuracy, the resulting graph for the classical RPP model tends to be very large. To overcome this difficulty, we proposed a two-phase heuristic algorithm that reduces the graph first and then solves the RPP on the reduced graph. We showed that the proposed two-phase heuristic has a performance guarantee of

TABLE 2. Results for **even** instances

| instance     | LB       | existing heuristic |                   | existing exact |                   | proposed    |                   | Gurobi      |                   |
|--------------|----------|--------------------|-------------------|----------------|-------------------|-------------|-------------------|-------------|-------------------|
|              |          | %gap               | time <sup>†</sup> | %gap           | time <sup>‡</sup> | %gap        | time <sup>‡</sup> | %gap        | time <sup>‡</sup> |
| DroneRPP56   | 5431.55  | 0.01               | 0.17              | <b>0.00</b>    | 4.54              | <b>0.00</b> | 0.31              | <b>0.00</b> | 7.74              |
| DroneRPP58   | 6602.73  | 0.04               | 1.02              | <b>0.00</b>    | 730.97            | <b>0.00</b> | 11.17             | <b>0.00</b> | 2340.35           |
| DroneRPP510  | 9608.49  | 0.03               | 1.05              | <b>0.00</b>    | 73.30             | <b>0.00</b> | 3.12              | <b>0.00</b> | 3047.43           |
| DroneRPP66   | 4861.05  | 0.04               | 1.25              | <b>0.00</b>    | 661.78            | <b>0.00</b> | 1.87              | <b>0.00</b> | 1026.47           |
| DroneRPP68   | 6899.64  | 0.15               | 3.55              | <b>0.00</b>    | 2164.14           | <b>0.00</b> | 13.92             | 0.39        | 2041.82           |
| DroneRPP610  | 9436.44  | 1.08               | 9.19              | t.l.           | t.l.              | <b>1.04</b> | 45.12             | 1.25        | 1645.07           |
| DroneRPP77   | 9656.78  | 0.10               | 1.92              | t.l.           | t.l.              | <b>0.05</b> | 1.60              | <b>0.05</b> | 3267.05           |
| DroneRPP79   | 12076.52 | 0.02               | 2.14              | <b>0.00</b>    | 1907.81           | <b>0.00</b> | 14.92             | 0.56        | 3552.19           |
| DroneRPP710  | 11416.28 | 0.06               | 2.86              | <b>0.00</b>    | 929.52            | <b>0.00</b> | 8.60              | 0.03        | 2553.69           |
| DroneRPP88   | 10068.64 | 0.02               | 2.55              | t.l.           | t.l.              | <b>0.00</b> | 21.56             | <b>0.00</b> | 667.88            |
| DroneRPP89   | 11236.67 | t.l.               | t.l.              | t.l.           | t.l.              | <b>0.82</b> | 2432.04           | 0.95        | 3318.21           |
| DroneRPP810  | 13681.12 | 0.76               | 4.91              | t.l.           | t.l.              | <b>0.68</b> | 64.71             | 0.74        | 3536.96           |
| DroneRPP99   | 12852.00 | 0.54               | 7.46              | t.l.           | t.l.              | <b>0.51</b> | 17.09             | 0.66        | 3342.83           |
| DroneRPP910  | 17156.69 | t.l.               | t.l.              | t.l.           | t.l.              | <b>1.35</b> | 630.82            | 2.45        | 2864.88           |
| DroneRPP1010 | 18157.81 | 0.85               | 336.21            | t.l.           | t.l.              | <b>0.81</b> | 58.11             | 2.16        | 2069.51           |

<sup>†</sup>The CPU time on a PC with an Intel Core i7 running at 3.4 GHz with 32 GB memory.

<sup>‡</sup>The CPU time on a PC with Xeon E-2286G CPU running at 4.00 GHz, with 64 GB memory, with computations always conducted on a single thread.

ratio 2. The experimental results showed that the proposed algorithm performed satisfactorily for all the tested instances, outperforming other existing algorithms.

**Acknowledgments.** This work was supported by JSPS KAKENHI [Grant No. 20H02388, 21K14367].

## REFERENCES

- [1] J. F. Campbell, Á. Corberán, I. Plana and J. M. Sanchis, [Drone arc routing problems](#), *Networks*, **72** (2018), 543-559.
- [2] J. F. Campbell, Á. Corberán, I. Plana, J. M. Sanchis and P. Segura, [Solving the length constrained  \$K\$ -drones rural postman problem](#), *European Journal of Operational Research*, **292** (2021), 60-72.
- [3] A. Corberán, I. Plana and J. M. Sanchis, [A branch & cut algorithm for the windy general routing problem and special cases](#), *Networks*, **49** (2007), 245-257.
- [4] H. A. Eiselt, M. Gendreau and G. Laporte, [Arc routing problems, part I: The Chinese postman problem](#), *Operations Research*, **43** (1995), 231-242.
- [5] H. A. Eiselt, M. Gendreau and G. Laporte, [Arc routing problems, part II: The rural postman problem](#), *Operations Research*, **43** (1995), 399-414.
- [6] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.
- [7] S. Poikonen and J. F. Campbell, [Future directions in drone routing research](#), *Networks*, **77** (2021), 116-126.

Received November 2022; revised October 2023; early access November 2023.

TABLE 3. Comparison between the existing heuristic algorithm and the proposed algorithm under the same environment for **random** instances

| instance      | existing heuristic |             |              | proposed  |             |              |
|---------------|--------------------|-------------|--------------|-----------|-------------|--------------|
|               | #vertices          | %gap        | time         | #vertices | %gap        | time         |
| DroneRPP561   | 76                 | 0.00        | 0.67         | 33        | <b>0.00</b> | <b>0.29</b>  |
| DroneRPP562   | 89                 | 0.00        | 0.36         | 30        | <b>0.00</b> | <b>0.27</b>  |
| DroneRPP581   | 109                | 0.00        | 0.94         | 89        | <b>0.00</b> | <b>0.65</b>  |
| DroneRPP582   | 139                | 0.00        | 1.08         | 24        | <b>0.00</b> | <b>0.56</b>  |
| DroneRPP5101  | 127                | 0.02        | 1.27         | 78        | <b>0.00</b> | <b>1.07</b>  |
| DroneRPP5102  | 191                | <b>0.00</b> | <b>0.92</b>  | 40        | 0.00        | 1.11         |
| DroneRPP661   | 89                 | 0.00        | 0.60         | 35        | <b>0.00</b> | <b>0.34</b>  |
| DroneRPP662   | 107                | <b>0.00</b> | <b>0.28</b>  | 30        | 0.00        | 0.40         |
| DroneRPP681   | 118                | 0.00        | 1.32         | 53        | <b>0.00</b> | <b>0.76</b>  |
| DroneRPP682   | 177                | 0.00        | 2.77         | 33        | <b>0.00</b> | <b>0.91</b>  |
| DroneRPP6101  | 169                | 0.01        | 1.91         | 135       | <b>0.00</b> | <b>9.63</b>  |
| DroneRPP6102  | 226                | 0.00        | 6.19         | 80        | <b>0.00</b> | <b>3.00</b>  |
| DroneRPP771   | 128                | <b>0.00</b> | <b>3.60</b>  | 94        | 0.00        | 5.07         |
| DroneRPP772   | 186                | 0.00        | 2.45         | 22        | <b>0.00</b> | <b>0.88</b>  |
| DroneRPP791   | 170                | <b>0.00</b> | <b>8.25</b>  | 125       | 0.00        | 8.35         |
| DroneRPP792   | 232                | 0.01        | 5.56         | 93        | <b>0.00</b> | <b>3.13</b>  |
| DroneRPP7101  | 211                | 0.01        | 10.26        | 126       | <b>0.00</b> | <b>4.95</b>  |
| DroneRPP7102  | 254                | 0.00        | 1.86         | 51        | <b>0.00</b> | <b>1.85</b>  |
| DroneRPP881   | 193                | <b>0.00</b> | <b>3.39</b>  | 149       | 0.00        | 3.88         |
| DroneRPP882   | 242                | 0.00        | 2.68         | 78        | <b>0.00</b> | <b>2.61</b>  |
| DroneRPP891   | 220                | 0.01        | 7.59         | 142       | <b>0.00</b> | <b>6.20</b>  |
| DroneRPP892   | 242                | 0.00        | 3.29         | 59        | <b>0.00</b> | <b>2.28</b>  |
| DroneRPP8101  | 237                | <b>0.66</b> | <b>18.39</b> | 169       | 0.66        | 33.68        |
| DroneRPP8102  | 302                | 0.00        | 14.77        | 71        | <b>0.00</b> | <b>3.27</b>  |
| DroneRPP991   | 232                | 0.00        | 8.40         | 109       | <b>0.00</b> | <b>6.05</b>  |
| DroneRPP992   | 308                | 0.00        | 11.60        | 87        | <b>0.00</b> | <b>3.81</b>  |
| DroneRPP9101  | 254                | <b>0.00</b> | <b>4.08</b>  | 193       | 0.00        | 6.04         |
| DroneRPP9102  | 315                | 0.01        | 36.21        | 192       | <b>0.00</b> | <b>16.59</b> |
| DroneRPP10101 | 281                | 0.00        | 6.29         | 185       | <b>0.00</b> | <b>6.18</b>  |
| DroneRPP10102 | 335                | 0.01        | 20.89        | 111       | <b>0.00</b> | <b>16.59</b> |

TABLE 4. Comparison between the existing heuristic algorithm and the proposed algorithm under the same environment for **even** instances

| instance     | existing heuristic |      |         | proposed  |             |                |
|--------------|--------------------|------|---------|-----------|-------------|----------------|
|              | #vertices          | %gap | time    | #vertices | %gap        | time           |
| DroneRPP56   | 94                 | 0.01 | 0.27    | 22        | <b>0.00</b> | <b>0.31</b>    |
| DroneRPP58   | 121                | 0.04 | 9.96    | 75        | <b>0.00</b> | <b>11.17</b>   |
| DroneRPP510  | 185                | 0.03 | 16.40   | 72        | <b>0.00</b> | <b>3.12</b>    |
| DroneRPP66   | 95                 | 0.04 | 4.86    | 39        | <b>0.00</b> | <b>1.87</b>    |
| DroneRPP68   | 130                | 0.15 | 65.52   | 51        | <b>0.00</b> | <b>13.92</b>   |
| DroneRPP610  | 175                | 1.08 | 78.64   | 108       | <b>1.04</b> | <b>45.12</b>   |
| DroneRPP77   | 177                | 0.10 | 9.09    | 38        | <b>0.05</b> | <b>1.60</b>    |
| DroneRPP79   | 232                | 0.02 | 19.60   | 112       | <b>0.00</b> | <b>14.92</b>   |
| DroneRPP710  | 220                | 0.06 | 14.86   | 134       | <b>0.00</b> | <b>8.60</b>    |
| DroneRPP88   | 193                | 0.02 | 19.60   | 169       | <b>0.00</b> | <b>21.56</b>   |
| DroneRPP89   | 226                | 0.95 | 1266.41 | 144       | <b>0.82</b> | <b>2432.04</b> |
| DroneRPP810  | 270                | 0.76 | 78.21   | 132       | <b>0.68</b> | <b>64.71</b>   |
| DroneRPP99   | 247                | 0.54 | 23.17   | 138       | <b>0.51</b> | <b>17.09</b>   |
| DroneRPP910  | 339                | 1.38 | 3571.23 | 152       | <b>1.35</b> | <b>630.82</b>  |
| DroneRPP1010 | 359                | 0.85 | 128.94  | 155       | <b>0.81</b> | <b>58.11</b>   |